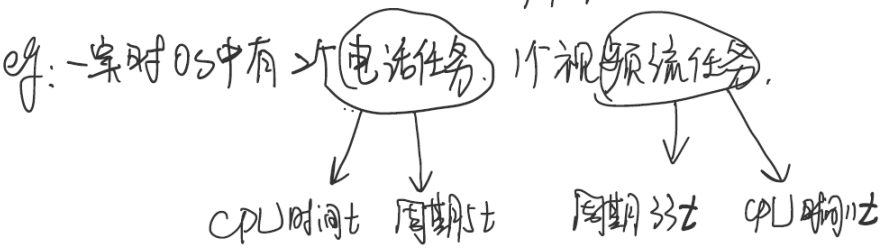
0. 判断实时系统可调度

(1) 有m个周期性进程

(2) CPU处理每个进程所时间为C_i，进程的周期时间为P_i

(3) 实时系统要可调度，应满足：$\sum_{i=1}^{m} \frac{C_i}{P_i} \le 1$



则 $\sum_{i=1}^{3} \frac{C_i}{P_i} = \frac{1}{5} + \frac{1}{5} + \frac{11}{33} = \frac{11}{15} < 1 \Rightarrow$ 可调度

1. 进程管理

1.1. 进程调度

	A	B	C	D	E
t	2	4	6	8	10
优先级	1	2	3	4	5 (5为最高优先级)

PS: 一均时间片t=2.

FCFS 顺序为 CDBEA，计算各自平均周转时间&带权周转时间

(1) FCFS:

顺序	C	D	B	E	A
执行时间	6	8	4	10	2
等待时间	0	6	6+8	6+8+4	6+8+4+10
	↓+6	↓+8	↓+4	↓+10	↓+2
周转	6	14	18	28	30

→ 共周转96、平均19.2.

(2) 优先级

顺序	E	D	C	B	A
执行时间	10	8	6	4	2
等待时间	0	10	10+8	10+8+6	10+8+6+4
	↓+10	↓+8	↓+6	↓+4	↓+2
	10	18	24	28	30

→ 共周转110、平均22

(3) SJF

顺序	A	B	C	D	E
执行时间	2	4	6	8	10
等待时间	0	2	6	12	20
周转时间	2	6	12	20	30.

→ 共周转70、平均14

(4) 时间片

顺序	A	B	C	D	E
Round 1	0	2	4	6	8
2	0	0	2	4	6
3	0	0	0	2	4
4	0	0	0	0	2
5	0	0	0	0	0

A结束于1轮第1片 → $t_A=2$.
B结束于2轮第1片 → $t_B=6\times2=12$.
C结束于2轮第1片 → $t_C=10\times2=20$.
D结束于4轮第1片 → $t_D=13\times2=26$.
E结束于5轮第1片 → $t_E=14\times2=28$

→ 共88 平均17.6.

1.2. 信号量计算

n个进程访问一互斥段，m<n，求信号量的范围。

最多m个进程

信号量>0 → 获得资源的进程数，<0 → 有多少在等。

(1) 有且仅有m个进程请求并访问互斥段时，信号量最大为m。

(2) 当所有n个进程都请求时，只容纳m个，剩下n-m个等待。

故信号量最小为 $-(n-m)=m-n$

$\Rightarrow m-n<a<m$。

1.3. 死锁的判断

1万个确认帐机，每个进程最多占4台，最多几个进程同时运行不死锁？

3个进程，4-4-4，不死锁

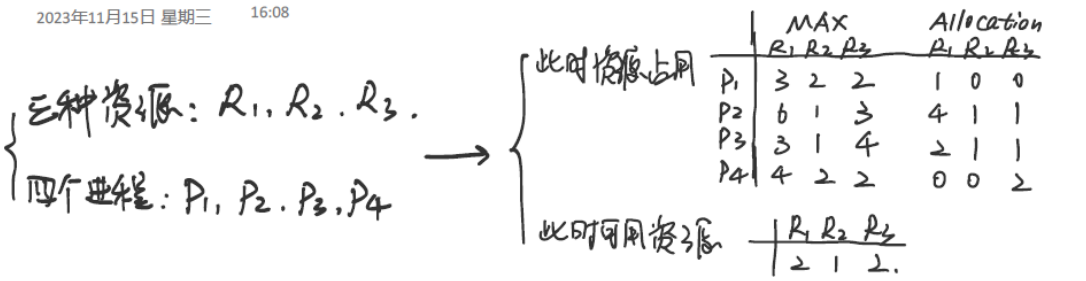
4个进程：3-3-3-4，占4个资源的执行完释放

后也不会死锁　　怎么会都有一个占3个资源

5个进程：3-3-3-3-1，死锁了。

最多4个

1.4. 银行家算法



(1) 表示出所有有关数据结构.

$Available = [2,1,2]$. $Max = \begin{bmatrix} 3&2&2 \\ 6&1&3 \\ 3&1&4 \\ 4&2&2 \end{bmatrix}$. $Allocation = \begin{bmatrix} 1&0&0 \\ 4&1&1 \\ 2&1&1 \\ 0&0&2 \end{bmatrix}$

$\Rightarrow Need = Max - Allocation = \begin{bmatrix} 2&2&2 \\ 2&0&2 \\ 1&0&3 \\ 4&2&0 \end{bmatrix}$

(2) 此时 P_1 发出请求资源 $Request = [1,0,1]$，能否安全满足此分配?

不能. 因为 $Need_{P_1} < Request$，故不能让 P_1 进程执行下去.

而其它进程也无法执行下去，故会死锁.

安全的分配策略为:

先分给 P_2 $\Rightarrow Allocation = \begin{bmatrix} 1&0&0 \\ 6&1&3 \\ 2&1&1 \\ 0&0&2 \end{bmatrix}$ $Available = [0,1,0]$

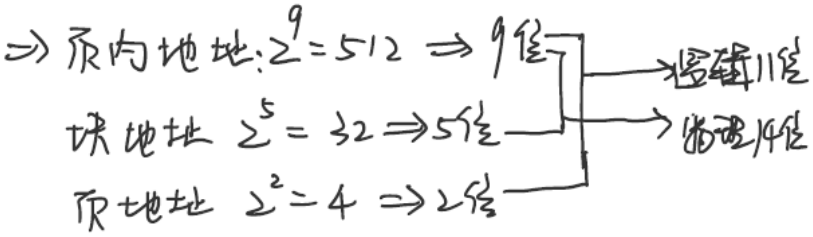
P_2 执行完释放 $Allocation = \begin{bmatrix} 1&0&0 \\ 0&0&0 \\ 2&1&1 \\ 0&0&2 \end{bmatrix}$ $Available = [6,2,3]$.. $Need = \begin{bmatrix} 2&2&2 \\ 0&0&0 \\ 1&0&3 \\ 4&2&0 \end{bmatrix}$

此时 $Allocation > \forall Need_{P_i}$. 故且使如何分配都不会死锁.

3. 内存管理

3.1. 地址位数

4页 → 32块，每页/块 512字节.



3.2. 逻辑-物理地址转换

进程页表

页号	状态位	访问位	修改位	块号
0	1	1	0	4
1	1	1	1	7
2	0	0	0	—
3	1	0	0	2
4	0	0	0	—
5	1	0	1	0

页大小为1024字节，求以下两逻辑地址的物理地址：5579，2232（十进制）

PS：
- 状态位：页表条目是否有效，不指向任何物理块
- 访问位：为1时表示页表被访问过。
- 修改位：为1时表示页面被写入/修改过。

$5579 = 5 \times 1024 + 459$ ⟶ 地址为 (5, 459) ⟶ (0, 459) → 459

$2232 = 2 \times 1024 + 184$ ⟶ 地址为 (2, 184) ⟶ Null.

3.3. 分页替换

请求分页：
- FIFO：先进先出。
- LRU：淘汰最少被使用的页。

T1：页面走向 5 0 1 2 0 3 0 4 2 3 0 3 2 0 1，内存中给该作业有3块。

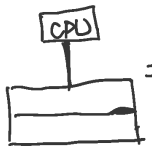
FIFO	5	0	1	2	0	3	0	4	2	3	0	3	2	0	1
内存	5	0	1	2	2	3	0	4	2	3	0	0	0	0	1
		5	0	1	1	2	3	0	4	2	3	3	3	3	0
			5	0	0	1	2	3	0	4	2	2	2	2	3
缺页?	√	√	√	√	X	√	√	√	√	√	√	X	X	X	√

⇒缺页73.3%

LRU	5	0	1	2	0	3	0	4	2	3	0	3	2	0	
内存	5	0	1	2	2	2	2	4	4	4	0	0	0	0	0
		5	0	1	1	3	3	3	2	2	2	2	2	2	2
			5	0	0	0	0	0	0	3	3	3	3	3	1
缺页?	√	√	√	√	X	√	X	√	√	√	√	X	X	X	√

⇒缺页66.7%

T2：



外存

a	b
c	d
x	y
z	t

页请求序：a b a c a b d b a c d

(1)

FIFO	a	b	a	c	a	b	d	b	a	c	d
内存	a	b	b	c	a	b	d	d	a	c	d
		a	a	b	c	a	b	b	d	a	c
缺页?	√	√	X	√	√	√	√	X	√	√	√

缺页率：$\frac{9}{11}$

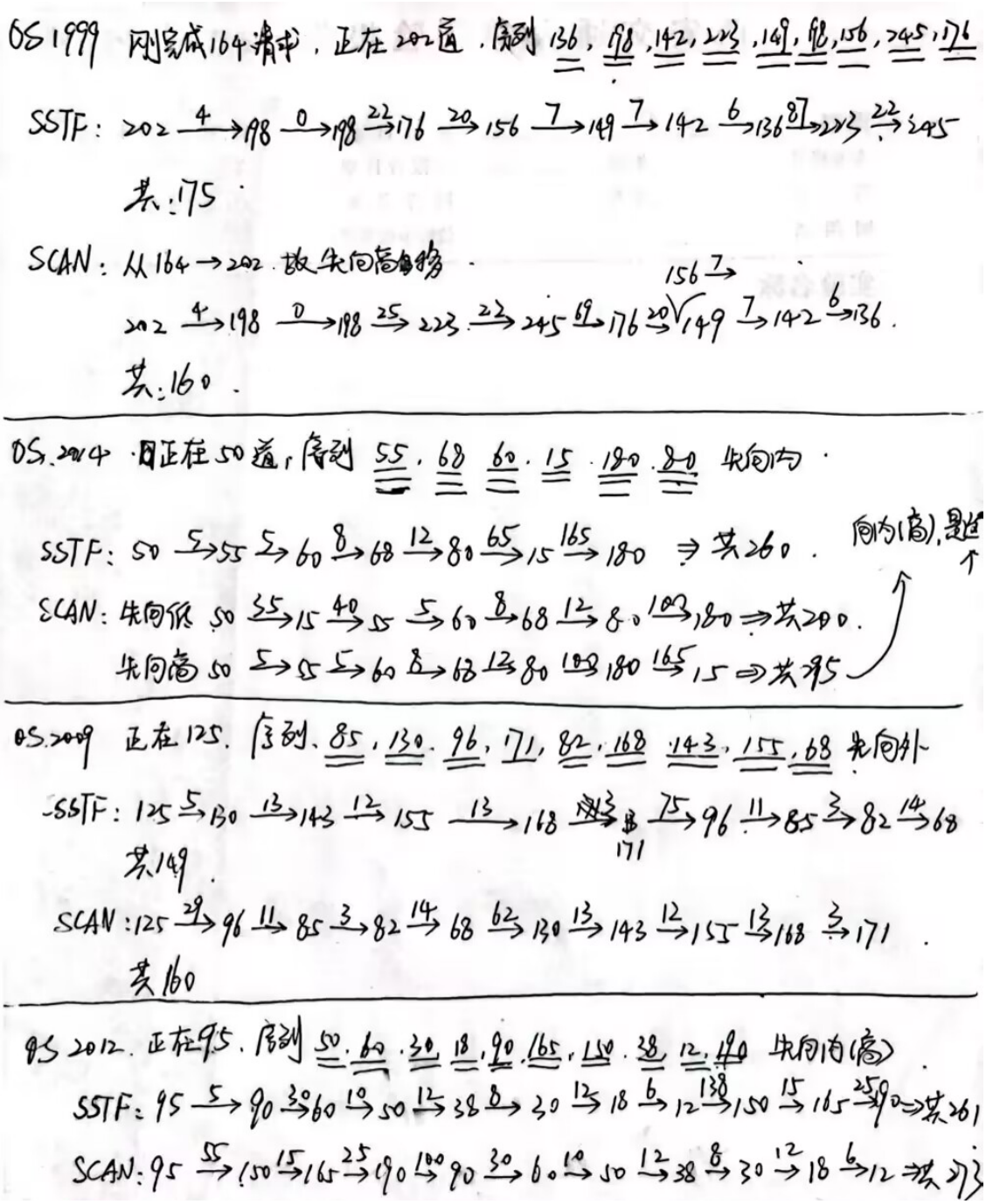
(2)

LRU	a	b	a	c	a	b	d	b	a	c	d
内存	a	b	a	c	a	b	d	b	a	c	d
		a	b	a	c	a	b	d	b	a	c
缺页?	√	√	X	√	X	√	√	X	√	√	√

缺页率：$\frac{8}{11}$

(3) 两种算法缺页率不同的原因：
- FIFO：完全按页面载入顺序淘汰
- LRU：考虑页面使用率淘汰，更好地反映了最近页面访问情况，故缺页率更低。

4. 磁盘调度



5. PV操作

5.1. Pre

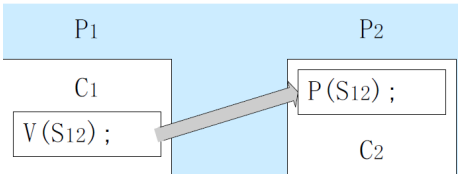
1 互斥信号量:

1. 为临界资源设置一个互斥信号量 mutex=1
2. 每个进程的临界区代码置于P(mutex) 和V(mutex) 原语之间

```
P(mutex);
  critical section
V(mutex);
  remainder section
```

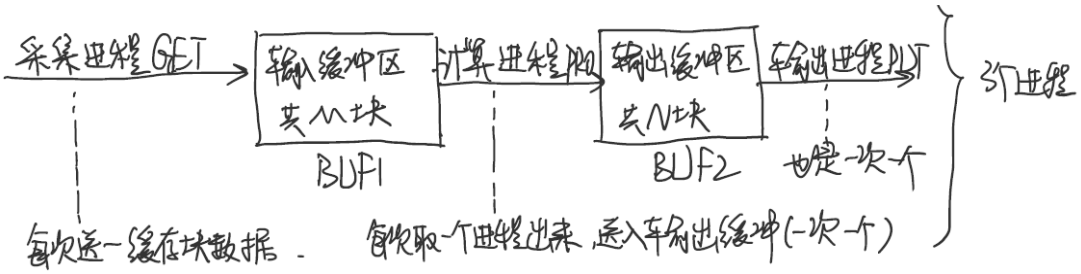
2 前驱关系

1. P1，P2并发执行，要求代码C1要在代码C2开始前完成
2. 为该前驱关系设置一个信号量S12=0



5.2. 生产者-消费者问题1

题目



解析

1. 信号量设置.

mutex1 / mutex2：确保对 M，N 缓冲区的互斥访问.

full1 / full2：记录 M.N 缓冲区的占用块数，初值为 0/0.

avail1 / avail2：记录 M.N 缓冲区的空闲块数，初值为 m/n.

2. 明确读者或/写者会的操作

L1：放进程.

P(avail) // 取走一个空块.

P(mutex) // 进入区，

// 数据放入Buff

V(mutex)

V(full) // 退出区，给个满块

goto：L1 // 反复执行，模拟进程.

L2：提取数据.

P(full) // 取走个满块.

P(mutex) // 互斥锁.

// 从Buff中取走数据.

V(mutex)

V(avail) // 当回个空块

P可理解为取，P(avail)为取走一个空闲块，P(full)为取走一满块.

V可理解为给，V(avail)为给一个空闲块，V(full)为给一个满块.

3. 求解本问题.

车到入进程为生产者——车入.

计算进程为消费者——先车到出再车到入.

车到出进程为消费者——车到出.

L1：采集进程

P(avail1)

P(mutex1)

// 数据放入Buff1

V(mutex1)

V(full1)

goto：L1

L2：计算进程.

P(full1)

P(mutex1)

车到入 // 从Buff1中取出

V(mutex1)

V(avail1)

P(avail2)

P(mutex2)

车到出 // 数据放入Buff2

V(mutex)

V(full2)

goto：L2

L3：车到出进程.

P(full2)

P(mutex2)

// 从Buff2取出.

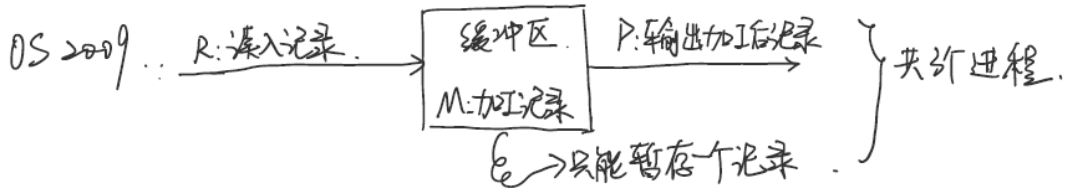
V(mutex2)

V(avail2)

goto L2

5.3. 生产者-消费者问题2

题目



解析

1. 信号量设置

S_1: 是否可将信息放入缓冲, 初值为1

S_2: 是否可在缓冲区中加工数据, 初值为0

S_3: 是否可把数据从缓存读出, 初值为0

2. 解题.

L_1: 读记录.
P(S_1)
//放入记录.
V(S_2)
初始: L_1

L_2: 处理记录.
P(S_2)
//加工记录.
V(S_3)
初始: L_2.

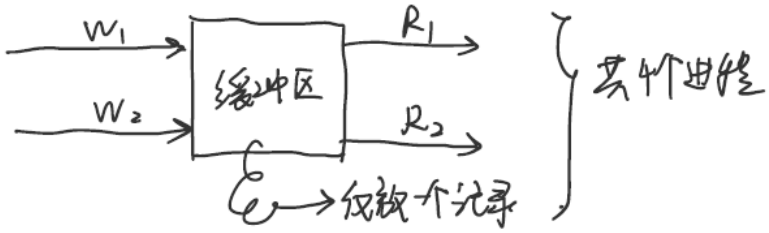
L_3: 输出
P(S_3)
//输出记录.
V(S_1).
初始: L_3.

3. Tips.

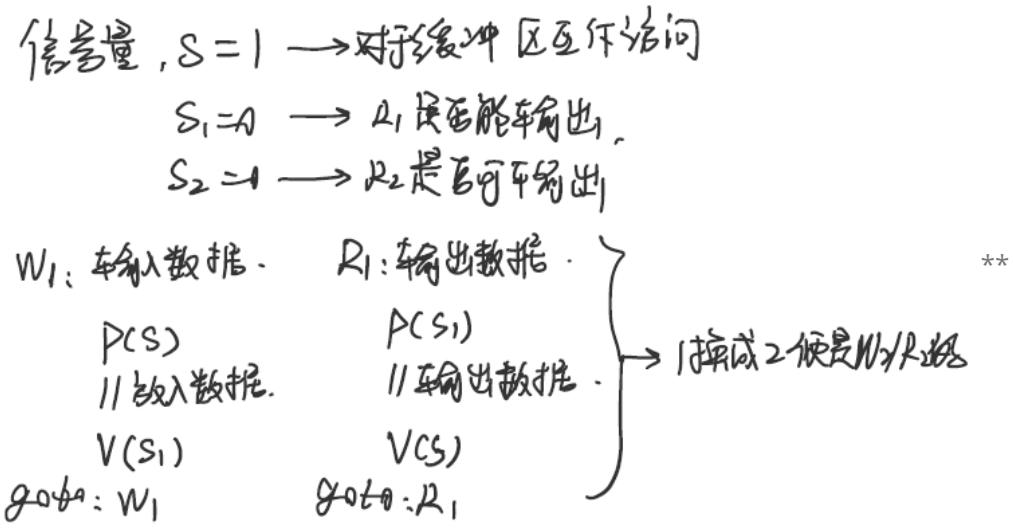
PV 在同一进程中为互斥, 在不同进程中为同步

5.4. 生产者-消费者问题3

题目



解析



**

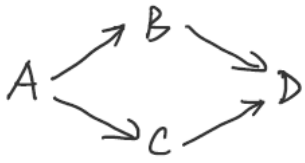
5.5. 利用进程前驱图问题

题目

1. 进程A每读入一批数据后把数据分解成二个可并发计算的数据块
2. 进程B和C分别处理两个分出来的数据块，BC处理涉及的是同一数据结构
3. 当进程B和C均完成处理后，再由进程D善后

解析

1. 画出进程关系图



2. 解析

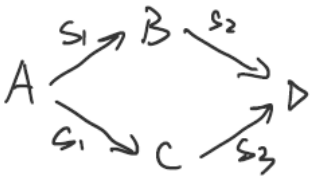
(1) 用点表示进程，线表示原语。

(2) 进程关系图中 ↓A 代表P操作，↑A 代表V操作，进程在PV间

例如 $\xrightarrow{b} S_3 \overset{e}{\underset{f}{\nearrow\searrow}}$ 表示为 {P(b); S_3; V(e); V(f)}

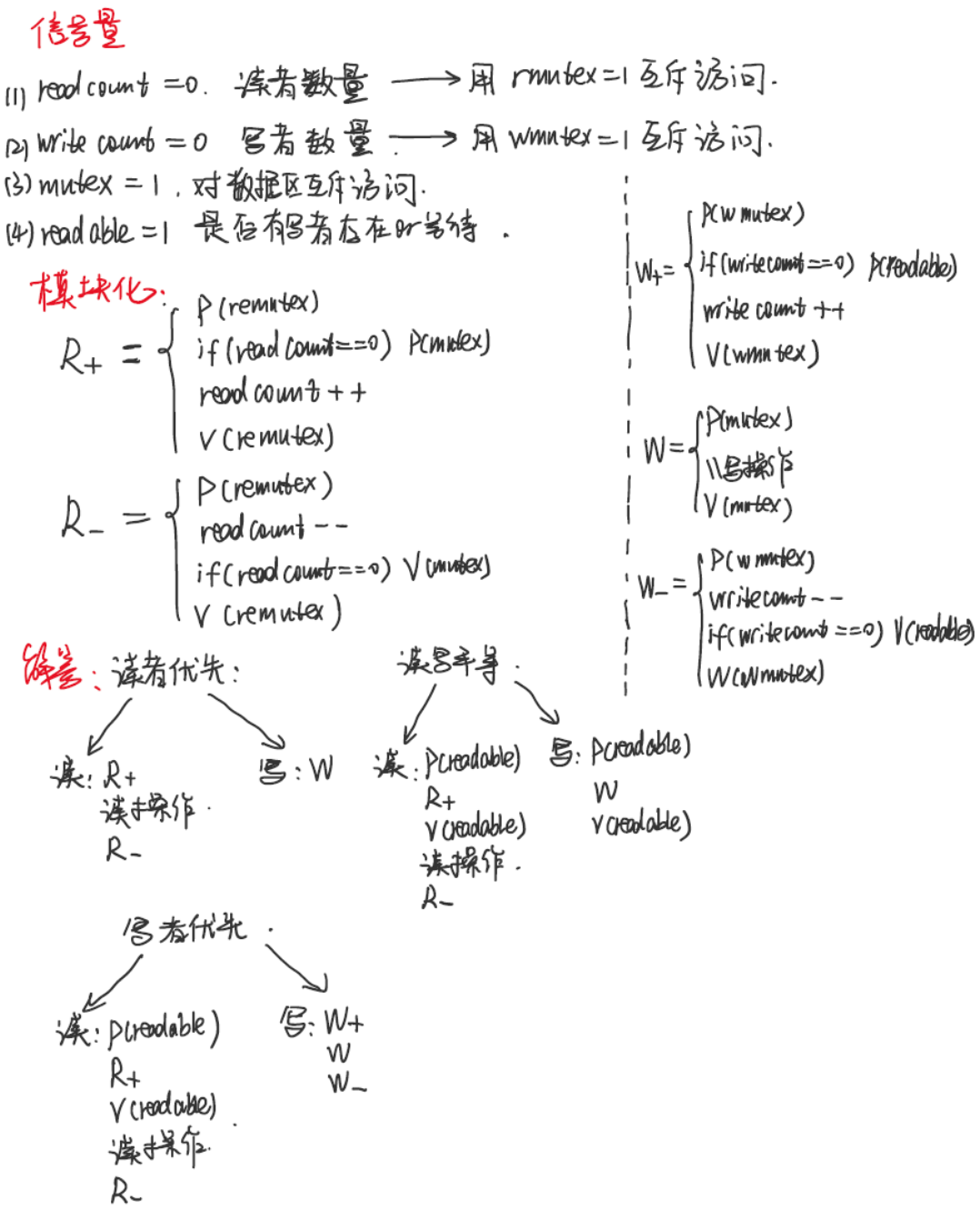
3. 解答

设置原语 S_1、S_2、S_3，初值为0。




```
par-begin
    BEGIN  A V(S1) V(S1) END.
    BEGIN  P(S1) B  V(S2) END.
    BEGIN  P(S1) C  V(S3) END.
    BEGIN  P(S2) P(S3)  D  END.
par-end .
```

5.6. 读者写者问题


信号量
(1) readcount = 0. 读者数量 → 用 rmutex = 1 互斥访问.
(2) writecount = 0 写者数量 → 用 wmutex = 1 互斥访问.
(3) mutex = 1. 对数据区互斥访问.
(4) readable = 1 是否有写者在or等待.
模块化:
R+ = { P(rmutex); if(readcount==0) P(mutex); readcount++; V(rmutex) }
R- = { P(rmutex); readcount--; if(readcount==0) V(mutex); V(rmutex) }
W+ = { P(wmutex); if(writecount==0) P(readable); writecount++; V(wmutex) }
W = { P(mutex); 写操作; V(mutex) }
W- = { P(wmutex); writecount--; if(writecount==0) V(readable); V(wmutex) }
读者优先:
读: R+ 读操作 R-
写: W
读写平等:
读: P(readable) R+ V(readable) 读操作 R-
写: P(readable) W V(readable)
写者优先:
读: P(readable) R+ V(readable) 读操作 R-
写: W+ W W-